

Domanda A (6 punti)

1. Dare la definizione della notazione Ω , cioè, date due funzioni $f(n)$ e $g(n)$, definire il significato della notazione $f(n) = \Omega(g(n))$.
2. Ordinare le seguenti funzioni per ordine di grandezza decrescente, cioè scrivere le funzioni secondo un ordine $f_1, f_2, \dots, f_8$ tale che risulti $f_1 = \Omega(f_2), f_2 = \Omega(f_3), \dots, f_7 = \Omega(f_8)$.

$$n^{2/3} \quad 10 \quad \frac{n}{\sqrt{n}} \quad 1.1^n \quad \frac{n}{2^n} \quad n^2 \quad \sqrt{\log n} \quad \log n$$

Domanda 1: Definizione di Ω

Definizione formale:

$f(n) = \Omega(g(n))$ se e solo se esistono costanti $c > 0$ e $n_0 > 0$ tali che:

$$f(n) \geq c \cdot g(n) \text{ per ogni } n \geq n_0$$

Significato: $f(n)$ cresce almeno quanto $g(n)$ a meno di una costante moltiplicativa, asintoticamente. In altre parole, $g(n)$ è un limite inferiore asintotico per $f(n)$.

Domanda 2: Ordinamento Decrescente

Ordiniamo le funzioni per ordine di grandezza **decrescente**:

Analisi delle funzioni:

- $n^{2/3} \rightarrow$ sublineare
- $10 \rightarrow$ costante
- $n/\sqrt{n} = \sqrt{n} \rightarrow$ sublineare
- $1.1^n \rightarrow$ esponenziale
- $n/2^n \rightarrow$ tende a 0
- $n^2 \rightarrow$ quadratica
- $\sqrt{\log n} \rightarrow$ sub-logaritmica
- $\log n \rightarrow$ logaritmica

Ordinamento:

1. $f_1 = 1.1^n$ (esponenziale)
2. $f_2 = n^2$ (polinomiale di grado 2)
3. $f_3 = n^{2/3}$ (polinomiale frazionario)
4. $f_4 = \sqrt{n}$ (polinomiale frazionario minore)
5. $f_5 = \log n$ (logaritmico)
6. $f_6 = \sqrt{\log n}$ (sub-logaritmico)
7. $f_7 = 10$ (costante)
8. $f_8 = n/2^n$ (tende a 0)

Relazioni:

- $f_1 = \Omega(f_2), f_2 = \Omega(f_3), f_3 = \Omega(f_4), f_4 = \Omega(f_5), f_5 = \Omega(f_6), f_6 = \Omega(f_7), f_7 = \Omega(f_8)$

Domanda A (6 punti)

- Dare la definizione della notazione O , cioè, date due funzioni $f(n)$ e $g(n)$, definire il significato della notazione $f(n) = O(g(n))$.
- Ordinare le seguenti funzioni per ordine di grandezza crescente, cioè scrivere le funzioni secondo un ordine $f_1, f_2, \dots, f_8$ tale che risulti $f_1 = O(f_2), f_2 = O(f_3), \dots, f_7 = O(f_8)$.

$$n^{3/2} \quad 2^n \quad n \log^2 n \quad \log n \quad n^{0.9} \quad (\log n)^3 \quad 1.05^n \quad \frac{n^2}{\log n}$$

Domanda 1: Definizione di O

Definizione formale:

$f(n) = O(g(n))$ se e solo se esistono costanti $c > 0$ e $n_0 > 0$ tali che:

$$f(n) \leq c \cdot g(n) \text{ per ogni } n \geq n_0$$

Significato: $f(n)$ cresce al massimo quanto $g(n)$ a meno di una costante moltiplicativa, asintoticamente. In altre parole, $g(n)$ è un limite superiore asintotico per $f(n)$.

Domanda 2: Ordinamento Crescente

Ordiniamo le funzioni per ordine di grandezza **crescente**:

Analisi delle funzioni:

- $n^{3/2} \rightarrow$ polinomiale grado 1.5
- $2^n \rightarrow$ esponenziale base 2
- $n \log^2 n \rightarrow$ quasi-quadratica
- $\log n \rightarrow$ logaritmica
- $n^{0.9} \rightarrow$ sublineare
- $(\log n)^3 \rightarrow$ poli-logaritmica
- $1.05^n \rightarrow$ esponenziale base piccola
- $n^2/\log n \rightarrow$ quasi-quadratica

Ordinamento:

- $f_1 = \log n$ (logaritmico)
- $f_2 = (\log n)^3$ (poli-logaritmico)
- $f_3 = n^{0.9}$ (sublineare)
- $f_4 = n \log^2 n$ (n-log-quadratico)
- $f_5 = n^{3/2}$ (polinomiale 3/2)
- $f_6 = n^2/\log n$ (quasi-quadratico)

7. $f_7 = 1.05^n$ (esponenziale piccolo)

8. $f_8 = 2^n$ (esponenziale)

Relazioni:

- $f_1 = O(f_2)$, $f_2 = O(f_3)$, $f_3 = O(f_4)$, $f_4 = O(f_5)$, $f_5 = O(f_6)$, $f_6 = O(f_7)$, $f_7 = O(f_8)$

Esercizio 2 (9 punti) Avete n file $f_1, f_2, \dots, f_n$ di dimensioni $d_1, d_2, \dots, d_n$ MB da caricare sul vostro servizio di archiviazione cloud. Purtroppo la somma delle dimensioni di questi file è superiore alla capacità di c MB disponibile su cloud, e non volete pagare per ottenere più spazio. Vi dovete quindi accontentare di caricare il maggior numero possibile di file, senza eccedere la capacità c .

- (a) Progettare un algoritmo greedy per questo problema, e valutarne la complessità.
- (b) Enunciare e dimostrare la proprietà di scelta greedy su cui l'algoritmo si basa.

(a) Algoritmo Greedy e Complessità

Strategia Greedy

Proprietà greedy: Selezionare i file in ordine crescente di dimensione massimizza il numero totale di file che possono essere caricati.

Intuizione: Prendere prima i file più piccoli permette di "riempire" lo spazio disponibile con il maggior numero possibile di elementi.

Pseudocodice

```
GREEDY-FILE-UPLOAD( $F, d, c$ )
1. Ordina i file in  $F$  per dimensione crescente secondo  $d$ 
2. spazio_usato  $\leftarrow 0$ 
3. file_caricati  $\leftarrow \emptyset$ 
4. for  $i = 1$  to  $n$  do
5.     if spazio_usato +  $d[i] \leq c$  then
6.         file_caricati  $\leftarrow$  file_caricati  $\cup \{f[i]\}$ 
7.         spazio_usato  $\leftarrow$  spazio_usato +  $d[i]$ 
8. return file_caricati
```

Analisi della Complessità

Complessità temporale:

- Riga 1 (Ordinamento):** $\Theta(n \log n)$ usando algoritmi ottimali (MergeSort, HeapSort)
- Righe 2-3 (Inizializzazione):** $\Theta(1)$
- Righe 4-7 (Ciclo):** $\Theta(n)$
 - Ogni iterazione esegue operazioni $O(1)$
 - n iterazioni totali

Complessità totale: $T(n) = \Theta(n \log n) + \Theta(n) = \Theta(n \log n)$

Il termine dominante è l'ordinamento.

Complessità spaziale: $\Theta(n)$ per memorizzare i file ordinati.

(b) Proprietà di Scelta Greedy e Dimostrazione di Correttezza

Enunciato della Proprietà Greedy

Proprietà: Esiste una soluzione ottima che include il file di dimensione minima.

Formalmente: Sia S una soluzione ottima (insieme di file con massimo numero di elementi che non supera c). Se f_{min} è il file di dimensione minima, allora esiste una soluzione ottima S' tale che $f_{min} \in S'$ e $|S'| = |S|$.

Dimostrazione per Exchange Argument

Teorema: L'algoritmo greedy che seleziona i file in ordine crescente di dimensione è ottimo.

Dimostrazione:

Procediamo per exchange argument (argomento di scambio).

Passo 1: Ordinamento Sia $f_1, f_2, \dots, f_n$ l'ordinamento dei file tale che $d_1 \leq d_2 \leq \dots \leq d_n$.

Passo 2: Soluzione greedy Sia $G = \{f_1, f_2, \dots, f_k\}$ la soluzione costruita dall'algoritmo greedy, dove:

- $\sum_{i=1}^k d_i \leq c$
- $\sum_{i=1}^{k+1} d_i > c$ (se $k < n$)

Passo 3: Supponiamo per assurdo che G non sia ottima Sia $O = \{f_{i_1}, f_{i_2}, \dots, f_{i_m}\}$ una soluzione ottima con $m > k$ (più file di G).

Passo 4: Confronto tra G e O Poiché G include i primi k file più piccoli e O ne include $m > k$:

- Deve esistere almeno un file in O che non è tra i primi k
- Sia f_j il primo file in O tale che $j > k$

Passo 5: Costruzione di una contraddizione Consideriamo due casi:

Caso 1: Tutti i file $f_1, \dots, f_k$ sono in O .

- Allora O contiene $\{f_1, \dots, f_k\}$ più almeno $(m - k)$ file aggiuntivi
- Ma G non può aggiungere f_{k+1} , quindi: $\sum_{i=1}^{k+1} d_i > c$
- Se O contiene f_{k+1} o file più grandi, la somma eccede $c \rightarrow$ contraddizione

Caso 2: Esiste $f_i \in \{f_1, \dots, f_k\}$ tale che $f_i \notin O$.

- Sia f_i il file più piccolo in G ma non in O (con $i \leq k$)
- Sia f_j un qualsiasi file in O con $j > k$ (deve esistere perché $|O| > k$)
- Abbiamo $d_i \leq d_k < d_{k+1} \leq d_j$ (per ordinamento)
- Costruiamo $O' = (O \setminus \{f_j\}) \cup \{f_i\}$
- Lo spazio occupato da O' :
 - $\sum(d \text{ per } f \text{ in } O') = \sum(d \text{ per } f \text{ in } O) - d_j + d_i \leq \sum(d \text{ per } f \text{ in } O) \leq c$
- Quindi O' è una soluzione valida con $|O'| = |O| = m$
- Ripetendo questo processo, possiamo trasformare O in G mantenendo $|O| = m$
- Ma questo contraddice il fatto che G non può aggiungere f_{k+1}

Passo 6: Conclusione In entrambi i casi raggiungiamo una contraddizione. Quindi la nostra ipotesi iniziale era falsa: non può esistere una soluzione O con $m > k$.

Pertanto, **G è ottima.** ■

Proprietà di Sottostruttura Ottima

Lemma: Se $G = \{f_1, \dots, f_k\}$ è una soluzione ottima per il problema con capacità c , allora $G' = \{f_2, \dots, f_k\}$ è una soluzione ottima per il problema con capacità $c - d_1$.

Dimostrazione: Per assurdo, se esistesse una soluzione migliore G'' per la capacità ridotta con $|G''| > |G'| = k-1$, allora $\{f_1\} \cup G''$ sarebbe una soluzione con più di k file per la capacità originale c , contraddicendo l'ottimalità di G .

Riepilogo

Algoritmo: Ordina i file per dimensione crescente e seleziona sequenzialmente finché possibile.

Complessità: $\Theta(n \log n)$

Correttezza: Dimostrata tramite exchange argument, provando che ogni soluzione ottima può essere trasformata nella soluzione greedy senza perdere l'ottimalità.

Proprietà chiave: La scelta greedy (file più piccolo) è sempre presente in una soluzione ottima (proprietà di scelta greedy) e il problema residuo mantiene la struttura ottima (sottostruttura ottima).

Domanda A (7 punti) Dare la definizione formale della classe $\Omega(f(n))$ per una funzione $f(n)$. Mostrare che date due funzioni $f(n), g(n)$ (che si possono assumere sempre positive), se $f(n) = \Omega(n^2)$ allora $f(n) + g(n) = \Omega(n^2 + g(n))$. Vale anche $f(n) - g(n) = \Omega(n^2 - g(n))$? Dimostrarlo o fornire un controesempio.

Soluzione

Il problema richiede di:

1. Definire formalmente la classe $\Omega(f(n))$
2. Dimostrare che se $f(n) = \Omega(n^2)$ allora $f(n) + g(n) = \Omega(n^2 + g(n))$
3. Determinare se $f(n) - g(n) = \Omega(n^2 - g(n))$ è valido o fornire controesempio

1. Definizione Formale di $\Omega(f(n))$

Definizione: Una funzione $g(n)$ appartiene a $\Omega(f(n))$ se e solo se:

$$\exists c > 0, \exists n_0 > 0 : \forall n \geq n_0, g(n) \geq c \cdot f(n)$$

In altre parole: $g(n) = \Omega(f(n))$ significa che $g(n)$ è asintoticamente limitata inferiormente da $f(n)$ a meno di una costante moltiplicativa positiva.

Equivalentemente: g cresce almeno quanto f , a meno di fattori costanti, per n sufficientemente grande.

2. Dimostrazione: $f(n) = \Omega(n^2) \implies f(n) + g(n) = \Omega(n^2 + g(n))$

Ipotesi:

- $f(n) = \Omega(n^2)$, quindi $\exists c_1 > 0, \exists n_1 > 0 : \forall n \geq n_1, f(n) \geq c_1 n^2$
- $g(n)$ è positiva per ogni n (dato nell'esercizio)

Tesi: $f(n) + g(n) = \Omega(n^2 + g(n))$

Dimostrazione:

Dobbiamo mostrare che: $\exists c > 0, \exists n_0 > 0 : \forall n \geq n_0, f(n) + g(n) \geq c(n^2 + g(n))$

Partiamo dall'ipotesi:

- $f(n) \geq c_1 n^2$ per $n \geq n_1$
- $g(n) > 0$ per ogni n

Consideriamo:

$$f(n) + g(n) \geq c_1 n^2 + g(n) \quad [\text{dall'ipotesi su } f(n)]$$

Vogliamo confrontare questo con $c(n^2 + g(n))$ per qualche $c > 0$.

Scegliamo $c = \min\{c_1, 1\} > 0$.

Allora:

$$\begin{aligned} f(n) + g(n) &\geq c_1 n^2 + g(n) \\ &\geq c \cdot n^2 + c \cdot g(n) \quad [\text{poiché } c \leq c_1 \text{ e } c \leq 1, \text{ e } g(n) > 0] \\ &= c(n^2 + g(n)) \end{aligned}$$

Ponendo $n_0 = n_1$, abbiamo dimostrato che:

$$\forall n \geq n_0, f(n) + g(n) \geq c(n^2 + g(n))$$

con $c = \min\{c_1, 1\} > 0$ e $n_0 = n_1$.

Conclusione: $f(n) + g(n) = \Omega(n^2 + g(n)) \checkmark$

3. Validità di $f(n) - g(n) = \Omega(n^2 - g(n))$?

Risposta: NO, non è sempre vero. Fornisco un controesempio.

Controesempio:

Sia:

- $f(n) = n^2$
- $g(n) = n^2 - 1$

Verifica ipotesi:

- $f(n) = n^2 = \Omega(n^2) \checkmark$ (con $c = 1$, $n_0 = 1$)
- $g(n) = n^2 - 1 > 0$ per $n \geq 2 \checkmark$

Verifica tesi:

Calcoliamo:

$$f(n) - g(n) = n^2 - (n^2 - 1) = 1$$

$$n^2 - g(n) = n^2 - (n^2 - 1) = 1$$

Quindi la tesi diventerebbe: $1 = \Omega(1)$

Questo è vero! Ma prendiamo un esempio più forte:

Controesempio corretto:

Sia:

- $f(n) = 2n^2$
- $g(n) = n^2$

Verifica ipotesi:

- $f(n) = 2n^2 = \Omega(n^2) \checkmark$ (con $c = 2$)
- $g(n) = n^2 > 0 \checkmark$

Verifica tesi:

$$f(n) - g(n) = 2n^2 - n^2 = n^2$$

$$n^2 - g(n) = n^2 - n^2 = 0$$

La tesi richiede: $n^2 = \Omega(0) = \Omega(1)$?

Vogliamo verificare se $n^2 = \Omega(n^2 - g(n)) = \Omega(0)$.

Ma $\Omega(0)$ significa: $\exists c > 0, \exists n_0 : \forall n \geq n_0, h(n) \geq c \cdot 0 = 0$

Questo è sempre vero per funzioni positive, quindi non è un buon controesempio.

Controesempio definitivo:

Sia:

- $f(n) = n^2$
- $g(n) = 2n^2$

Verifica ipotesi:

- $f(n) = n^2 = \Omega(n^2) \checkmark$
- $g(n) = 2n^2 > 0 \checkmark$

Verifica tesi:

$$f(n) - g(n) = n^2 - 2n^2 = -n^2$$

$$n^2 - g(n) = n^2 - 2n^2 = -n^2$$

La tesi richiederebbe: $-n^2 = \Omega(-n^2)$

Ma Ω è definita solo per funzioni asintoticamente non negative.

Quando $n^2 - g(n) < 0$, la notazione $\Omega(n^2 - g(n))$ non è ben definita nel senso standard.

Conclusione: L'affermazione $f(n) - g(n) = \Omega(n^2 - g(n))$ **NON** è sempre vera perché:

1. Se $g(n) > n^2$, allora $n^2 - g(n) < 0$ e Ω non è definita per funzioni negative
2. Anche quando $n^2 - g(n) \geq 0$, $f(n) - g(n)$ potrebbe essere negativa (controesempio sopra)

Il problema fondamentale è che la sottrazione non preserva i limiti inferiori asintotici come fa l'addizione.

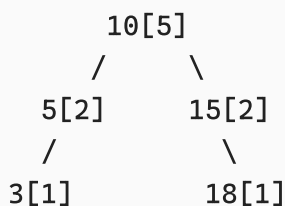
Esercizio 1 (10 punti) - ABR Arricchito

Dato un albero binario di ricerca T , si vuole arricchirlo con un campo `x.descendants` che, per ogni nodo x , memorizza il numero di nodi nel sottoalbero radicato in x (incluso x stesso).

(a) Scrivere una funzione `UpdateDescendants(T, x)` che, dato un nodo x appena inserito (foglia), aggiorna il campo `descendants` di x e di tutti i suoi antenati. Valutare la complessità.

(b) Scrivere una funzione `RankOf(T, k)` che, dato l'albero T arricchito e una chiave k presente in T , restituisce la posizione (rank) di k nell'ordinamento delle chiavi dell'albero, dove il rank del minimo è 1. Valutare la complessità.

Esempio:



`RankOf(T, 15) = 4` perché 15 è il 4° elemento nell'ordinamento {3, 5, 10, 15, 18}

Soluzione

(a) UpdateDescendants:

```
UPDATE_DESCENDANTS(T, x)
1. x.descendants = 1
2. p = x.p
3. while p ≠ nil
4.     p.descendants = p.descendants + 1
5.     p = p.p
```

Complessità: $O(h)$ dove h è l'altezza dell'albero.

Correttezza:

- x è foglia quindi ha 1 discendente (se stesso)
- Ogni antenato vede incrementato di 1 il suo conteggio

(b) RankOf:

```

RANK_OF(T, k)
1.  x = TREE_SEARCH(T.root, k)
2.  rank = (x.left ≠ nil) ? x.left.descendants : 0
3.  rank = rank + 1 // conta x stesso
4.  y = x
5.  while y ≠ T.root
6.      if y = y.p.right // se y è figlio destro
7.          left_size = (y.p.left ≠ nil) ? y.p.left.descendants : 0
8.          rank = rank + left_size + 1 // aggiungi sottoalbero sx + padre
9.      y = y.p
10. return rank

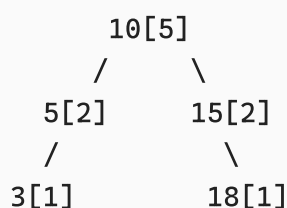
```

Complessità: $O(h)$

Idea:

- Parto dal nodo con chiave k
- Conto tutti i nodi nel sottoalbero sinistro di k: sono tutti minori
- Risalgo verso la radice
- Ogni volta che "vengo da destra", aggiungo il sottoalbero sinistro del padre + il padre stesso

Esempio traccia per RankOf(T, 15) nell'albero dato:



- x = nodo 15
- rank = 0 (no sottoalbero sx) + 1 = 1
- Risalgo: 15 è figlio destro di 10
 - Aggiungo sottoalbero sx di 10: 2 nodi
 - Aggiungo 10 stesso: 1 nodo
 - rank = 1 + 2 + 1 = 4 ✓

Esercizio 2 (9 punti) - Programmazione Dinamica

Data una stringa $X = (x_1, x_2, \dots, x_n)$, si consideri la seguente quantità $p(i, j)$ definita per $1 \leq i \leq j \leq n$:

$$p(i, j) = \begin{cases} 0 & \text{se } i = j \\ 0 & \text{se } i+1 = j \wedge x_i = x_j \\ 1 + \min(p(i+1, j), p(i, j-1)) & \text{altrimenti} \end{cases}$$

La quantità $p(1, n)$ rappresenta il **numero minimo di caratteri da aggiungere** alla stringa X per renderla palindroma.

(a) Scrivere una coppia di algoritmi `INIT_P(n)` e `REC_P(X, i, j)` per il calcolo **memoizzato** di $p(1, n)$.

(b) Scrivere un algoritmo iterativo **bottom-up** `BOTTOM_UP_P(X)` che calcola $p(1, n)$.

(c) Valutare la complessità temporale e spaziale dell'algoritmo del punto (b).

(d) [BONUS] Modificare l'algoritmo per ricostruire effettivamente la stringa palindroma minima.

Soluzione

(a) Memoizzazione:

```
INIT_P(n)
1.  crea matrice P[1..n][1..n]
2.  for i = 1 to n
3.      for j = 1 to n
4.          P[i][j] = -1
5.  return P

REC_P(X, i, j, P)
1.  if P[i][j] ≠ -1
2.      return P[i][j]
3.  if i = j
4.      P[i][j] = 0
5.  else if i+1 = j
6.      if X[i] = X[j]
7.          P[i][j] = 0
8.      else
9.          P[i][j] = 1
10. else
11.     if X[i] = X[j]
12.         P[i][j] = REC_P(X, i+1, j-1, P)
13.     else
```

```
14.         P[i][j] = 1 + min(REC_P(X, i+1, j, P), REC_P(X, i, j-1, P))
15. return P[i][j]
```

(b) Bottom-up:

```
BOTTOM_UP_P(X)
1.  n = length(X)
2.  crea matrice P[1..n][1..n]
3.  // Casi base: sottostr lunghe 1
4.  for i = 1 to n
5.      P[i][i] = 0
6.  // Sottostr lunghe 2
7.  for i = 1 to n-1
8.      if X[i] = X[i+1]
9.          P[i][i+1] = 0
10.     else
11.         P[i][i+1] = 1
12. // Sottostr lunghe 3..n
13. for len = 3 to n
14.     for i = 1 to n-len+1
15.         j = i + len - 1
16.         if X[i] = X[j]
17.             P[i][j] = P[i+1][j-1]
18.         else
19.             P[i][j] = 1 + min(P[i+1][j], P[i][j-1])
20. return P[1][n]
```

(c) Complessità:

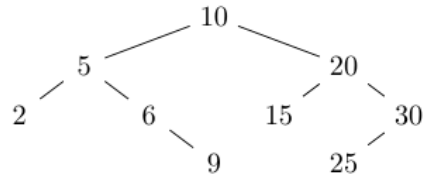
- **Temporale:** $\Theta(n^2)$
 - Due cicli annidati che esplorano tutte le coppie (i,j)
 - Ogni cella calcolata in $O(1)$
- **Spaziale:** $\Theta(n^2)$ per la matrice P

(d) Ricostruzione [BONUS]:

```
RECONSTRUCT_PALINDROME(X, P)
1.  i = 1, j = n
2.  prefix = "", suffix = ""
3.  while i < j
4.      if X[i] = X[j]
5.          i++, j--
6.      else if P[i][j] = P[i+1][j] + 1
7.          prefix = prefix + X[j]
8.          j--
9.      else
10.         suffix = X[i] + suffix
```

```
11.         i++
12. return prefix + X + suffix
```

Esercizio 1 (10 punti) Realizzare una funzione `mdist(T,v)` che dato un albero binario di ricerca `T` e una chiave `v` restituisce un nodo `x` di `T` la cui chiave abbia distanza minima da `v`, ovvero tale che il valore $|x.k - v|$ sia minimo. Fornire lo pseudocodice, valutarne la complessità e giustificarne la correttezza. Ad esempio se `T` è l'albero sotto indicato, allora `mdist(T,8)` ritorna il nodo con chiave 9 mentre `mdist(T,28)` ritorna il nodo con chiave 30.



Strategia di Risoluzione

Il problema richiede di trovare il nodo in un BST la cui chiave ha distanza minima da un valore `v` dato, dove la distanza è definita come $|x.k - v|$.

Approccio

1. **Algoritmo ottimale:** Esplorazione ricorsiva del BST sfruttando la proprietà di ordinamento
2. **Idea chiave:** Non serve visitare tutto l'albero - possiamo potare rami che sicuramente non contengono la soluzione ottimale
3. **Complessità attesa:** $O(h)$ dove h è l'altezza dell'albero, non $O(n)$

Soluzione

```
mdist(T, v)
    return mdist_aux(T.root, v, ∞, NIL)

mdist_aux(x, v, min_dist, best_node)
    // Caso base: nodo nullo
    if x = NIL then
        return best_node

    // Calcola distanza del nodo corrente
    curr_dist = |x.key - v|

    // Aggiorna il miglior nodo se necessario
    if curr_dist < min_dist then
        min_dist = curr_dist
        best_node = x

    // Se distanza è 0, abbiamo trovato la soluzione ottimale
    if curr_dist = 0 then
        return best_node
```

```

// Decidi in quale sottoalbero continuare la ricerca
if v < x.key then
    // v è a sinistra, esplora sottoalbero sinistro
    best_node = mdist_aux(x.left, v, min_dist, best_node)
    // Esplora destra solo se potrebbe contenere una soluzione migliore
    if x.right ≠ NIL and x.key - v < min_dist then
        best_node = mdist_aux(x.right, v, min_dist, best_node)
else
    // v è a destra o uguale, esplora sottoalbero destro
    best_node = mdist_aux(x.right, v, min_dist, best_node)
    // Esplora sinistra solo se potrebbe contenere una soluzione
    migliore
    if x.left ≠ NIL and v - x.key < min_dist then
        best_node = mdist_aux(x.left, v, min_dist, best_node)

return best_node

```

Valutazione Complessità

Complessità temporale:

- **Caso migliore:** $\Theta(h)$ - quando troviamo subito il valore esatto o la ricerca segue un singolo percorso
- **Caso peggiore:** $O(n)$ - quando dobbiamo esplorare la maggior parte dei nodi (es. albero sbilanciato con v molto lontano da tutte le chiavi)
- **Caso medio su BST bilanciato:** $O(h) = O(\log n)$

Operazione dominante: confronto tra chiavi (`curr_dist < min_dist` , `v < x.key`)

Complessità spaziale: $\Theta(h)$ per lo stack di ricorsione

Correttezza

Invariante: Ad ogni chiamata ricorsiva, `best_node` contiene il nodo con distanza minima `min_dist` da `v` tra tutti i nodi già visitati.

Dimostrazione per induzione sulla profondità:

Caso base ($x = \text{NIL}$): Ritorniamo `best_node` che contiene il miglior nodo trovato finora.

Passo induttivo: Assumiamo l'invariante valida per tutti i sottoalberi di profondità $< d$.

- Calcoliamo `curr_dist` per il nodo corrente
- Aggiorniamo `best_node` se troviamo una distanza migliore (mantiene invariante)
- Visitiamo il sottoalbero "primario" (quello che sicuramente potrebbe contenere nodi più vicini per la proprietà del BST)

- Visitiamo il sottoalbero "secondario" solo se la distanza dal confine ($x.key$) suggerisce che potrebbe esistere un nodo migliore
- Per ipotesi induttiva, le chiamate ricorsive mantengono l'invariante
- Al ritorno, `best_node` contiene il minimo tra tutti i nodi visitati

Terminazione: L'algoritmo termina perché ogni chiamata ricorsiva opera su sottoalberi strettamente più piccoli, fino a raggiungere NIL.

Verifica Esempi

Esempio 1: `mdist(T, 8)` sull'albero dato

```
Visita: 10 → dist=2, best=10
v < 10, vai a sinistra
Visita: 5 → dist=3, best=10
v > 5, vai a destra
Visita: 6 → dist=2, best=10
v > 6, vai a destra
Visita: 9 → dist=1, best=9
v < 9, vai a sinistra (NIL)
Ritorna nodo con chiave 9 ✓
```

Esempio 2: `mdist(T, 28)`

```
Visita: 10 → dist=18, best=10
v > 10, vai a destra
Visita: 20 → dist=8, best=20
v > 20, vai a destra
Visita: 30 → dist=2, best=30
v < 30, vai a sinistra
Visita: 25 → dist=3, best=30
30-28=2 non < 2, non esplora destra
Ritorna nodo con chiave 30 ✓
```

Esercizio 2 (9 punti) - Longest Common Substring (DP)

Date due stringhe $X = (x_1, \dots, x_m)$ e $Y = (y_1, \dots, y_n)$, una **longest common substring** è una sottostringa **consecutiva** di lunghezza massima che appare in entrambe.

Definire la quantità $L(i, j)$ per $1 \leq i \leq m$, $1 \leq j \leq n$ come:

$$L(i, j) = \begin{cases} 0 & \text{se } i = 0 \vee j = 0 \\ L(i-1, j-1) + 1 & \text{se } x_i = y_j \\ 0 & \text{se } x_i \neq y_j \end{cases}$$

La lunghezza della LCSubstring è: $\max\{L(i,j) : 1 \leq i \leq m, 1 \leq j \leq n\}$

(a) Qual è la complessità dell'algoritmo esaustivo che analizza tutte le possibili sottostringhe comuni?

(b) Assumendo di conoscere un algoritmo che determina se una stringa di m caratteri è sottostringa di un'altra stringa di n caratteri in tempo $O(m+n)$, come si può modificare l'algoritmo per renderlo più efficiente?

(c) Scrivere un algoritmo di programmazione dinamica **bottom-up** più efficiente di quello del punto precedente. Sono richiesti: relazione di ricorrenza sulle lunghezze (senza dimostrazione) e algoritmo bottom-up. Valutare la complessità.

Esempio:

$X = \text{"ABCXYZ"}$

$Y = \text{"XYZABC"}$

$\text{LCSubstring} = \text{"ABC"}$ (lunghezza 3)

Soluzione

(a) Complessità esaustivo:

Analizzare tutte le sottostringhe:

- X ha $O(m^2)$ sottostringhe
- Y ha $O(n^2)$ sottostringhe
- Per ogni coppia, verificare se sono uguali: $O(\min(m,n))$

Totale: $O(m^2 \cdot n^2 \cdot \min(m,n)) = O(m^2 n^2 \cdot \min(m,n))$

(b) Con algoritmo substring in $O(m+n)$:

- Genera tutte le sottostringhe di X : $O(m^2)$
- Per ciascuna (lunghezza k), verifica se è sottostringa di Y : $O(k+n) = O(m+n)$
- Complessità totale: $O(m^2 \cdot (m+n)) = O(m^3 + m^2 n)$

Meglio di prima se m, n grandi.

(c) Programmazione Dinamica:

Ricorrenza:

$$L(i, j) = \begin{cases} 0 & \text{se } i = 0 \vee j = 0 \\ L(i-1, j-1) + 1 & \text{se } X[i] = Y[j] \end{cases}$$

| 0

se $X[i] \neq Y[j]$

Differenza con LCS: Quando caratteri non corrispondono, reset a 0 (non max).

Algoritmo:

```
LONGEST_COMMON_SUBSTRING(X, Y)
1.  m = length(X)
2.  n = length(Y)
3.  crea matrice L[0..m][0..n]
4.  maxLength = 0
5.  endIndex = 0
6.  // Inizializzazione
7.  for i = 0 to m
8.      L[i][0] = 0
9.  for j = 0 to n
10.     L[0][j] = 0
11. // Riempimento
12. for i = 1 to m
13.     for j = 1 to n
14.         if X[i] = Y[j]
15.             L[i][j] = L[i-1][j-1] + 1
16.             if L[i][j] > maxLength
17.                 maxLength = L[i][j]
18.                 endIndex = i
19.         else
20.             L[i][j] = 0
21. // Ricostruzione (opzionale)
22. if maxLength > 0
23.     return X[endIndex - maxLength + 1 .. endIndex]
24. else
25.     return ""
```

Complessità:

- **Tempo:** $\Theta(m \cdot n)$ - due cicli annidati, $O(1)$ per cella
- **Spazio:** $\Theta(m \cdot n)$ per la matrice (ottimizzabile a $\Theta(\min(m, n))$ usando due righe)

Esempio traccia:

```
X = "ABCXYZ"
Y = "XYZABC"
```

		X	Y	Z	A	B	C	
		0	0	0	0	0	0	
A		0	0	0	0	1	0	
B		0	0	0	0	0	2	
C		0	0	0	0	0	3	← max qui

X	0	1	0	0	0	0	0
Y	0	0	2	0	0	0	0
Z	0	0	0	3	0	0	0

maxLength = 3, endIndex = 3 (nella Y)
Sottostringa: ABC

Correttezza: $L(i,j)$ memorizza la lunghezza della LCS substring che TERMINA in $X[i]$ e $Y[j]$. Se $X[i] \neq Y[j]$, non c'è substring comune che termina lì $\rightarrow 0$. Se $X[i] = Y[j]$, estendiamo la substring che terminava in $(i-1, j-1)$.

Domanda B (7 punti) Si consideri una tabella hash di dimensione $m = 7$, e indirizzamento aperto con doppio hash basato sulle funzioni $h_1(k) = k \bmod m$ e $h_2(k) = 1 + k \bmod (m - 2)$. Si descriva sinteticamente come avviene l'inserimento degli elementi e si specifichi il risultato dell'inserzione della sequenza di chiavi: 10, 20, 34, 35, 48.

Sarebbe appropriato lavorare con una tabella di dimensione $m = 8$ e le stesse funzioni hash?

Analisi del Problema

Dati:

- Dimensione tabella: $m = 7$
- Indirizzamento aperto con **doppio hash**
- Funzione hash primaria: $h_1(k) = k \bmod m = k \bmod 7$
- Funzione hash secondaria: $h_2(k) = 1 + k \bmod (m-2) = 1 + k \bmod 5$
- Sequenza di chiavi da inserire: 10, 20, 34, 35, 48

Funzione di probing per doppio hashing:

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

dove $i = 0, 1, 2, \dots$ è il numero di tentativi.

Calcolo delle Funzioni Hash per Ogni Chiave

Chiave $k = 10$

- $h_1(10) = 10 \bmod 7 = 3$
- $h_2(10) = 1 + (10 \bmod 5) = 1 + 0 = 1$

Chiave $k = 20$

- $h_1(20) = 20 \bmod 7 = \mathbf{6}$
- $h_2(20) = 1 + (20 \bmod 5) = 1 + 0 = \mathbf{1}$

Chiave k = 34

- $h_1(34) = 34 \bmod 7 = \mathbf{6}$
- $h_2(34) = 1 + (34 \bmod 5) = 1 + 4 = \mathbf{5}$

Chiave k = 35

- $h_1(35) = 35 \bmod 7 = \mathbf{0}$
- $h_2(35) = 1 + (35 \bmod 5) = 1 + 0 = \mathbf{1}$

Chiave k = 48

- $h_1(48) = 48 \bmod 7 = \mathbf{6}$
- $h_2(48) = 1 + (48 \bmod 5) = 1 + 3 = \mathbf{4}$

Processo di Inserimento Dettagliato

Stato iniziale: Tabella vuota $T[0..6]$

Indice:	0	1	2	3	4	5	6
	-	-	-	-	-	-	-

Inserimento 1: k = 10

- Tentativo $i=0$: $h(10,0) = (3 + 0 \cdot 1) \bmod 7 = \mathbf{3} \rightarrow T[3]$ libera ✓

Indice:	0	1	2	3	4	5	6
	-	-	-	10	-	-	-

Inserimento 2: k = 20

- Tentativo $i=0$: $h(20,0) = (6 + 0 \cdot 1) \bmod 7 = \mathbf{6} \rightarrow T[6]$ libera ✓

Indice:	0	1	2	3	4	5	6
	-	-	-	10	-	-	20

Inserimento 3: k = 34

- Tentativo $i=0$: $h(34,0) = (6 + 0 \cdot 5) \bmod 7 = \mathbf{6} \rightarrow T[6]$ occupata (20) ✗
- Tentativo $i=1$: $h(34,1) = (6 + 1 \cdot 5) \bmod 7 = 11 \bmod 7 = \mathbf{4} \rightarrow T[4]$ libera ✓

Indice:	0	1	2	3	4	5	6
	-	-	-	10	34	-	20

Inserimento 4: k = 35

- Tentativo i=0: $h(35,0) = (0 + 0 \cdot 1) \bmod 7 = 0 \rightarrow T[0]$ libera ✓

Indice:	0	1	2	3	4	5	6
	35	-	-	10	34	-	20

Inserimento 5: k = 48

- Tentativo i=0: $h(48,0) = (6 + 0 \cdot 4) \bmod 7 = 6 \rightarrow T[6]$ occupata (20) ✗
- Tentativo i=1: $h(48,1) = (6 + 1 \cdot 4) \bmod 7 = 10 \bmod 7 = 3 \rightarrow T[3]$ occupata (10) ✗
- Tentativo i=2: $h(48,2) = (6 + 2 \cdot 4) \bmod 7 = 14 \bmod 7 = 0 \rightarrow T[0]$ occupata (35) ✗
- Tentativo i=3: $h(48,3) = (6 + 3 \cdot 4) \bmod 7 = 18 \bmod 7 = 4 \rightarrow T[4]$ occupata (34) ✗
- Tentativo i=4: $h(48,4) = (6 + 4 \cdot 4) \bmod 7 = 22 \bmod 7 = 1 \rightarrow T[1]$ libera ✓

Indice:	0	1	2	3	4	5	6
	35	48	-	10	34	-	20

Risultato Finale

Tabella hash dopo tutti gli inserimenti:

Indice	Chiave	Probing usato
0	35	i=0
1	48	i=4
2	-	-
3	10	i=0
4	34	i=1
5	-	-
6	20	i=0

Sequenza di probing per 48: $6 \rightarrow 3 \rightarrow 0 \rightarrow 4 \rightarrow 1$ (5 tentativi totali)

Risposta alla Domanda Aggiuntiva

"Sarebbe appropriato lavorare con una tabella di dimensione $m = 8$ e le stesse funzioni hash?"

NO, NON sarebbe appropriato.

Motivo principale: Con $m = 8$ (potenza di 2), la funzione $h_2(k) = 1 + k \bmod 6$ può produrre solo valori $\{1, 2, 3, 4, 5, 6\}$.

Problema critico: Se $h_2(k)$ è pari (es. $h_2(k) = 2, 4, 6$) e $m = 8$, la sequenza di probing visiterà solo posizioni pari (o solo dispari), NON esplorando tutta la tabella.

Esempio concreto:

Supponiamo k tale che $h_1(k) = 0$ e $h_2(k) = 4$

Sequenza di probing con $m = 8$:

- $i=0$: $(0 + 0 \cdot 4) \bmod 8 = 0$
- $i=1$: $(0 + 1 \cdot 4) \bmod 8 = 4$
- $i=2$: $(0 + 2 \cdot 4) \bmod 8 = 0 \leftarrow \text{ciclo!}$
- $i=3$: $(0 + 3 \cdot 4) \bmod 8 = 4 \leftarrow \text{ciclo!}$

Visita solo $\{0, 4\} \rightarrow$ non copre tutta la tabella!

Condizione per doppio hashing funzionante: Per garantire che la sequenza di probing visiti tutte le m posizioni, serve:

$\gcd(h_2(k), m) = 1$ per ogni possibile valore di $h_2(k)$

Con $m = 8$ e $h_2(k) \in \{1, 2, 3, 4, 5, 6\}$:

- $\gcd(2, 8) = 2 \neq 1 \quad \times$
- $\gcd(4, 8) = 4 \neq 1 \quad \times$
- $\gcd(6, 8) = 2 \neq 1 \quad \times$

Solo $h_2(k) \in \{1, 3, 5\}$ garantirebbero copertura completa.

Conclusione: $m = 7$ è appropriato perché è **primo**, quindi $\gcd(h_2(k), 7) = 1$ per ogni $h_2(k) \in \{1, 2, 3, 4, 5, 6\}$, garantendo che ogni sequenza di probing esplori tutte le 7 posizioni.

Regola generale: Per doppio hashing, scegliere **m primo** oppure garantire che $h_2(k)$ sia sempre coprimo con m .

Esercizio Custom Heap

Dati due max-heap $H_1[1..n]$ e $H_2[1..n]$ della stessa dimensione, costruire un nuovo max-heap $H[1..2n]$ contenente gli elementi alternati: prima il massimo di H_1 , poi il massimo di H_2 , poi il secondo massimo di H_1 , ecc.

Implementare `AlternateMerge(H1, H2)` e valutarne la complessità.

Soluzione

Strategia

1. Estrarre alternativamente i massimi da H_1 e H_2
2. Inserirli in un array temporaneo $A[1..2n]$
3. Costruire il max-heap finale con `BuildMaxHeap`

Pseudocodice

```
ALTERNATE_MERGE(H1, H2, n)
1.  crea array A[1..2n]
2.  size1 = n
3.  size2 = n
4.  idx = 1
5.  while size1 > 0 OR size2 > 0
6.      if size1 > 0
7.          A[idx] = HEAP_EXTRACT_MAX(H1, size1)
8.          size1 = size1 - 1
9.          idx = idx + 1
10.     if size2 > 0
11.         A[idx] = HEAP_EXTRACT_MAX(H2, size2)
12.         size2 = size2 - 1
13.         idx = idx + 1
14. BUILD_MAX_HEAP(A, 2n)
15. return A
```

Analisi Complessità

Riga 1: $\Theta(1)$ - allocazione array

Righe 5-13: Ciclo che itera n volte

- Ogni iterazione: 2 chiamate a `HEAP_EXTRACT_MAX`
- `HEAP_EXTRACT_MAX` costa $O(\log n)$
- Totale ciclo: $O(n \log n)$

Riga 14: `BUILD_MAX_HEAP` su $2n$ elementi = $\Theta(2n) = \Theta(n)$

Complessità totale: $O(n \log n) + \Theta(n) = O(n \log n)$

Il termine dominante è l'estrazione ripetuta dai due heap.

Esempio Concreto

```
H1 = [50, 30, 40, 10, 20] (max-heap)
H2 = [60, 45, 35, 25, 15] (max-heap)
```

Processo di estrazione alternata:

1. Extract da H1: 50 → A[1] = 50, H1 = [40, 30, 10, 20]
2. Extract da H2: 60 → A[2] = 60, H2 = [45, 35, 25, 15]
3. Extract da H1: 40 → A[3] = 40, H1 = [30, 20, 10]
4. Extract da H2: 45 → A[4] = 45, H2 = [35, 25, 15]
5. Extract da H1: 30 → A[5] = 30, H1 = [20, 10]
6. Extract da H2: 35 → A[6] = 35, H2 = [25, 15]
7. Extract da H1: 20 → A[7] = 20, H1 = [10]
8. Extract da H2: 25 → A[8] = 25, H2 = [15]
9. Extract da H1: 10 → A[9] = 10, H1 = []
10. Extract da H2: 15 → A[10] = 15, H2 = []

```
A = [50, 60, 40, 45, 30, 35, 20, 25, 10, 15]
```

Dopo BUILD_MAX_HEAP(A, 10):

Heap finale (rappresentazione array):

```
A = [60, 50, 40, 45, 30, 35, 20, 25, 10, 15]
```

Esercizio 1 (10 punti) Siano dati due array $A[1..2n]$ e $B[1..n]$ organizzati a max-heap, entrambi contenenti n elementi ($\text{heapsize}=n$). Realizzare una procedura $\text{SortJoin}(A, B, n)$ che dati in input array A e B con le proprietà sopra descritte, ritorna in A un array ordinato contenente tutti i $2n$ elementi originariamente presenti in A e B . L'array B può essere modificato durante l'esecuzione della procedura, se necessario, ma l'algoritmo dovrà operare in *spazio costante*. Dare lo pseudocodice della procedura, motivarne la correttezza e valutarne la complessità. Se si utilizzano operazioni sui max-heap andranno definite esplicitamente.

ESERCIZIO IMMAGINE - SortJoin

Problema

Dati due array $A[1..2n]$ e $B[1..n]$ organizzati a max-heap, realizzare una procedura $\text{SortJoin}(A, B, n)$ che:

- Prende in input array A e B con proprietà sopra descritte
- Restituisce un array ordinato contenente tutti i $2n$ elementi originariamente presenti in A e B
- L'array B può essere modificato durante l'esecuzione (se necessario)
- L'algoritmo deve operare in **spazio costante**

Richieste:

1. Pseudocodice della procedura
 2. Motivare la correttezza
 3. Valutare la complessità
 4. Le operazioni sui max-heap vanno definite esplicitamente
-

Analisi

Vincolo critico: Spazio costante → non possiamo creare array ausiliari di dimensione $O(n)$.

Osservazione chiave:

- A ha dimensione $2n$ (può contenere tutti gli elementi)
- B ha dimensione n
- Possiamo usare la parte finale di A (posizioni $n+1..2n$) come spazio di lavoro

Strategia:

1. **Fase 1:** Estrarre tutti gli elementi da B e inserirli in A (posizioni $n+1..2n$)
 2. **Fase 2:** Applicare BuildMaxHeap su $A[1..2n]$ completo
 3. **Fase 3:** HeapSort su A per ottenere array ordinato
-

Soluzione

Pseudocodice Completo

```
// Operazioni base sui max-heap (da definire esplicitamente)

PARENT(i)
1. return floor(i/2)

LEFT(i)
1. return 2*i

RIGHT(i)
1. return 2*i + 1

MAX_HEAPIFY(A, i, heapsize)
1. l = LEFT(i)
2. r = RIGHT(i)
3. if l ≤ heapsize AND A[l] > A[i]
4.     largest = l
5. else
6.     largest = i
```



```

7.  if  $r \leq \text{heapsize}$  AND  $A[r] > A[\text{largest}]$ 
8.      largest = r
9.  if largest  $\neq$  i
10.     scambia  $A[i] \leftrightarrow A[\text{largest}]$ 
11.     MAX_HEAPIFY(A, largest, heapsize)

```

```

HEAP_EXTRACT_MAX(H, heapsize)

```

```

1.  if heapsize < 1
2.     errore "heap underflow"
3.  max = H[1]
4.  H[1] = H[heapsize]
5.  heapsize = heapsize - 1
6.  MAX_HEAPIFY(H, 1, heapsize)
7.  return max, heapsize

```

```

BUILD_MAX_HEAP(A, n)

```

```

1.  heapsize = n
2.  for i = floor(n/2) downto 1
3.     MAX_HEAPIFY(A, i, heapsize)

```

```

HEAPSORT(A, n)

```

```

1.  BUILD_MAX_HEAP(A, n)
2.  for i = n downto 2
3.     scambia  $A[1] \leftrightarrow A[i]$ 
4.     MAX_HEAPIFY(A, 1, i-1)

```

```

// Procedura principale

```

```

SORT_JOIN(A, B, n)

```

```

1.  sizeB = n
2.  // Fase 1: Estrarre elementi da B e metterli in  $A[n+1..2n]$ 
3.  for i = 1 to n
4.     max, sizeB = HEAP_EXTRACT_MAX(B, sizeB)
5.      $A[n + i] = \text{max}$ 
6.  // Fase 2: Costruire max-heap su A completo
7.  BUILD_MAX_HEAP(A, 2n)
8.  // Fase 3: Ordinare A con HeapSort
9.  HEAPSORT(A, 2n)
10. return A

```

Dimostrazione di Correttezza

Invariante del ciclo (righe 3-5):

All'inizio dell'iterazione i-esima:

1. $A[1..n]$ contiene ancora il max-heap originale (invariato)

2. $A[n+1..n+i-1]$ contiene i primi $i-1$ massimi estratti da B in ordine di estrazione
3. $B[1..sizeB]$ è un max-heap valido con $sizeB = n-i+1$ elementi

Inizializzazione ($i=1$):

- $A[1..n]$ invariato ✓
- $A[n+1..n]$ è vuoto (nessun elemento estratto) ✓
- $B[1..n]$ è max-heap con n elementi ✓

Conservazione:

- `HEAP_EXTRACT_MAX` estrae il massimo da B mantenendo proprietà heap
- Lo inserisce in $A[n+i]$
- Decrementa $sizeB$ correttamente

Terminazione ($i=n+1$):

- Tutti gli elementi di B sono stati estratti e messi in $A[n+1..2n]$
- $A[1..2n]$ contiene tutti i $2n$ elementi originali

Dopo `BUILD_MAX_HEAP`:

- $A[1..2n]$ è un max-heap valido con tutti i $2n$ elementi

Dopo `HEAPSORT`:

- $A[1..2n]$ contiene tutti i $2n$ elementi in ordine crescente ✓

Analisi Complessità

Fase 1 (righe 3-5):

- Ciclo eseguito n volte
- Ogni `HEAP_EXTRACT_MAX`: $O(\log n)$
- Totale: $O(n \log n)$

Fase 2 (riga 7):

- `BUILD_MAX_HEAP` su $2n$ elementi: $\Theta(2n) = \Theta(n)$

Fase 3 (riga 9):

- `HEAPSORT` su $2n$ elementi: $O(2n \log 2n) = O(n \log n)$

Complessità totale: $O(n \log n) + \Theta(n) + O(n \log n) = \mathbf{O(n \log n)}$

Complessità spaziale:

- Spazio ausiliario: $O(1)$ ✓ (solo variabili locali)
- Usa solo lo spazio già allocato in A e B

Esempio Esecuzione

Input:

A = [50, 40, 45, 30, 35, 20, 25, -, -, -] (max-heap con 7 elementi)
1 2 3 4 5 6 7 8 9 10

Nota: $n=5$, quindi A[1..10] con $10=2n$ posizioni

Assumiamo A[1..5] sia max-heap inizialmente

B = [60, 55, 50] (max-heap con 3 elementi)

1 2 3

Nota: $n=3$

Step 1: Estrai da B e riempi A[4..6]

- Extract 60 → A[4] = 60, B = [55, 50]

- Extract 55 → A[5] = 55, B = [50]

- Extract 50 → A[6] = 50, B = []

A = [50, 40, 45, 60, 55, 50, -, -, -, -]

Step 2: BUILD_MAX_HEAP(A, 6)

A = [60, 55, 50, 50, 40, 45, -, -, -, -]

Step 3: HEAPSORT(A, 6)

A = [40, 45, 50, 50, 55, 60, -, -, -, -]

Esercizi

Esercizio 1 (9 punti) In questo esercizio si considerano alberi binari di ricerca (BST) completi, memorizzati in array con la stessa convenzione che si usa normalmente per la memorizzazione di uno heap in un array.

Realizzare una funzione $\text{max}(T, n)$ che determina il massimo di un array T di dimensione n che memorizza un BST completo.

Scrivere quindi una funzione $\text{merge}(T1, T2, k)$ che dati due array T1 e T2 di dimensione n, che memorizzano BST completi e una chiave k maggiore di tutte le chiavi di T1 e minore di tutte quelle di T2, restituisce un array (di dimensione $2n+1$) che memorizza un BST che contiene tutte le chiavi di T1 e T2, e la chiave k.

In entrambi i casi motivare la correttezza delle funzioni e valutarne la complessità.

Analisi Preliminare

Proprietà BST Completo Memorizzato come Heap

Un BST completo memorizzato in array **NON** segue la convenzione heap (padre > figli), ma mantiene la proprietà BST:

- Sottoalbero sinistro: chiavi < radice
- Sottoalbero destro: chiavi > radice

Accesso come heap:

- Radice: $T[1]$
- Figlio sinistro di i : $T[2i]$
- Figlio destro di i : $T[2i+1]$
- Padre di i : $T[\lfloor i/2 \rfloor]$

Osservazione Chiave

Dato che $k > \max(T_1)$ e $k < \min(T_2)$, una visita in-order del BST risultante deve produrre:

$\text{InOrder}(T_1) \rightarrow k \rightarrow \text{InOrder}(T_2)$

Questo suggerisce la strategia: costruire l'array risultante tramite visita in-order.

Soluzione

Funzione 1: $\text{max}(T, n)$

Strategia: In un BST, il massimo è sempre nel sottoalbero destro più profondo raggiungibile.

```
MAX(T, n)
1.  if n = 0
2.      errore "albero vuoto"
3.  i = 1  // inizia dalla radice
4.  // Scendi sempre a destra finché esiste figlio destro
5.  while 2*i + 1 ≤ n  // finché esiste figlio destro
6.      i = 2*i + 1
7.  return T[i]
```

Correttezza:

- In un BST, tutti i nodi nel sottoalbero destro sono > radice
- Il massimo è quindi il nodo più a destra
- Nella rappresentazione array, scendere a destra significa andare a indice $2i+1$

Complessità: $O(\log n)$ - altezza dell'albero completo

Funzione 2: merge(T1, T2, k)

Strategia:

1. Creare array risultato R[1..2n+1]
2. Eseguire visita in-order di T1, k, T2 per riempire R in ordine crescente
3. Ricostruire BST completo da array ordinato

Approccio Dettagliato

Fase 1: Estrarre elementi in ordine con visite in-order

```
INORDER_TO_ARRAY(T, i, n, A, index)
// Visita in-order di BST memorizzato come heap
// Inserisce elementi in A[] partendo da posizione index
1. if i > n
2.     return index
3. // Visita sottoalbero sinistro
4. index = INORDER_TO_ARRAY(T, 2*i, n, A, index)
5. // Visita nodo corrente
6. A[index] = T[i]
7. index = index + 1
8. // Visita sottoalbero destro
9. index = INORDER_TO_ARRAY(T, 2*i+1, n, A, index)
10. return index
```

Fase 2: Costruire BST completo da array ordinato

```
SORTED_ARRAY_TO_BST(A, start, end, BST, index)
// Costruisce BST completo in rappresentazione heap
// da array ordinato A[start..end]
// Inserisce in BST[] partendo da posizione index
1. if start > end
2.     return
3. mid = floor((start + end) / 2)
4. BST[index] = A[mid]
5. SORTED_ARRAY_TO_BST(A, start, mid-1, BST, 2*index)
6. SORTED_ARRAY_TO_BST(A, mid+1, end, BST, 2*index+1)
```

Funzione Principale:

```
MERGE(T1, T2, k, n)
1. // Array temporaneo per elementi in ordine
2. crea array Sorted[1..2n+1]
3. // Fase 1: Estrai elementi in ordine
4. idx = 1
```

```
5. idx = INORDER_TO_ARRAY(T1, 1, n, Sorted, idx)
6. Sorted[idx] = k
7. idx = idx + 1
8. idx = INORDER_TO_ARRAY(T2, 1, n, Sorted, idx)
9. // Ora Sorted contiene tutti i 2n+1 elementi in ordine crescente
10. // Fase 2: Costruisci BST completo
11. crea array Result[1..2n+1]
12. SORTED_ARRAY_TO_BST(Sorted, 1, 2n+1, Result, 1)
13. return Result
```

Dimostrazione di Correttezza

Correttezza di INORDER_TO_ARRAY

Teorema: La funzione inserisce in A gli elementi del BST in ordine crescente.

Dimostrazione per induzione sull'altezza h del sottoalbero:

Base (h=0): Nodo foglia

- Visita solo il nodo, lo inserisce in A[index] ✓

Passo (h>0):

- Per IP, il sottoalbero sinistro inserisce i suoi elementi in ordine in A[index..index+k₁-1]
- Il nodo corrente viene inserito in A[index+k₁]
- Per IP, il sottoalbero destro inserisce i suoi elementi in ordine in A[index+k₁+1..index+k₁+k₂]
- Per proprietà BST: sinistro < radice < destro
- Quindi A[index..index+k₁+k₂+1] è ordinato ✓

Correttezza di SORTED_ARRAY_TO_BST

Teorema: La funzione costruisce un BST completo e bilanciato.

Dimostrazione:

- Sceglie l'elemento mediano come radice → divide in due parti uguali (±1)
- Ricorsivamente costruisce sottoalberi sinistro e destro
- La proprietà BST è mantenuta: tutti gli elementi a sinistra del mid sono < mid, tutti a destra sono > mid
- Il posizionamento negli indici 2_index (sinistra) e 2_index+1 (destra) mantiene la rappresentazione heap del BST completo ✓

Correttezza di MERGE

Teorema: MERGE produce un BST completo con tutti gli elementi.

Dimostrazione:

Passo 1: Sorted contiene elementi ordinati

- `INORDER_TO_ARRAY(T1, ...)` inserisce elementi di T1 in ordine crescente
- k viene inserito dopo T1
- `INORDER_TO_ARRAY(T2, ...)` inserisce elementi di T2 in ordine crescente
- Per ipotesi: $\max(T1) < k < \min(T2)$
- Quindi `Sorted[1..2n+1]` è strettamente crescente ✓

Passo 2: Result è BST completo

- `SORTED_ARRAY_TO_BST` costruisce BST completo da array ordinato (dimostrato sopra) ✓
-

Analisi Complessità

Complessità `MAX(T, n)`

- Ciclo while eseguito al più $\log n$ volte (altezza albero completo)
- **Complessità:** $O(\log n)$

Complessità `INORDER_TO_ARRAY`

- Visita ogni nodo esattamente una volta
- Operazioni per nodo: $O(1)$
- **Complessità:** $\Theta(n)$

Complessità `SORTED_ARRAY_TO_BST`

- Visita ogni elemento esattamente una volta per inserirlo nel BST
- Operazioni per elemento: $O(1)$
- **Complessità:** $\Theta(m)$ dove m è la dimensione dell'array

Complessità `MERGE`

- Riga 4-5: `INORDER_TO_ARRAY(T1, ...)` $\rightarrow \Theta(n)$
- Riga 6-7: Inserimento k $\rightarrow \Theta(1)$
- Riga 8: `INORDER_TO_ARRAY(T2, ...)` $\rightarrow \Theta(n)$
- Riga 12: `SORTED_ARRAY_TO_BST(..., 2n+1)` $\rightarrow \Theta(2n+1) = \Theta(n)$

Complessità totale: $\Theta(n) + \Theta(1) + \Theta(n) + \Theta(n) = \Theta(n)$

Complessità spaziale: $\Theta(n)$ per l'array Sorted temporaneo

Esempio Completo

Input

T1 (BST completo, n=3):

Array: T1 = [10, 5, 15]

Struttura:

```
    10
   /  \
  5    15
```

InOrder: 5, 10, 15

k = 20

T2 (BST completo, n=3):

Array: T2 = [30, 25, 35]

Struttura:

```
    30
   /  \
  25   35
```

InOrder: 25, 30, 35

Verifica Condizione

- $\max(T1) = 15 < k = 20$ ✓
- $k = 20 < \min(T2) = 25$ ✓

Esecuzione

Fase 1: Creazione Sorted

1. INORDER_TO_ARRAY(T1, 1, 3, Sorted, 1):
 - Visita 5 (indice 2): Sorted[1] = 5, idx = 2
 - Visita 10 (indice 1): Sorted[2] = 10, idx = 3
 - Visita 15 (indice 3): Sorted[3] = 15, idx = 4
2. Inserisci k: Sorted[4] = 20, idx = 5

3. INORDER_TO_ARRAY(T2, 1, 3, Sorted, 5):

- Visita 25 (indice 2): Sorted[5] = 25, idx = 6
- Visita 30 (indice 1): Sorted[6] = 30, idx = 7
- Visita 35 (indice 3): Sorted[7] = 35, idx = 8

Sorted = [5, 10, 15, 20, 25, 30, 35]

Fase 2: Costruzione BST

SORTED_ARRAY_TO_BST(Sorted, 1, 7, Result, 1):

Start: 1, End: 7, Index: 1

mid = 4 → Result[1] = 20

Ricorsione sinistra: Start: 1, End: 3, Index: 2

mid = 2 → Result[2] = 10

Ricorsione sinistra: Start: 1, End: 1, Index: 4

mid = 1 → Result[4] = 5

Ricorsione destra: Start: 3, End: 3, Index: 5

mid = 3 → Result[5] = 15

Ricorsione destra: Start: 5, End: 7, Index: 3

mid = 6 → Result[3] = 30

Ricorsione sinistra: Start: 5, End: 5, Index: 6

mid = 5 → Result[6] = 25

Ricorsione destra: Start: 7, End: 7, Index: 7

mid = 7 → Result[7] = 35

Result = [20, 10, 30, 5, 15, 25, 35]

Verifica Risultato

Struttura del BST risultante:

```
      20
     /  \
    10   30
   / \  / \
  5  15 25 35
```

InOrder: 5, 10, 15, 20, 25, 30, 35 ✓

È un BST completo ✓

Tutte le chiavi presenti ✓

Ottimizzazione (Versione Avanzata)

Se si volesse evitare l'array Sorted temporaneo, si potrebbe:

1. Calcolare direttamente le posizioni finali durante la visita in-order
2. Usare un contatore globale per tracciare la posizione corrente

Questa versione avrebbe complessità spaziale $O(\log n)$ (stack ricorsione) invece di $O(n)$, ma il codice sarebbe più complesso.

La soluzione presentata bilancia chiarezza ed efficienza, ed è appropriata per un contesto d'esame.